



United International University (UIU)
Department of Computer Science and Engineering (CSE)
Course: VLSI Design (CSE 4327)

Experiment 4: Design of a 4-bit Full Adder using Verilog

Objectives

1. To design and simulate a half adder using Verilog dataflow modeling.
2. To build a 1-bit full adder hierarchically by instantiating a half adder module.
3. To construct a 4-bit full adder hierarchically from 1-bit full adders using a generate block.
4. To extend the design to a 32-bit full adder (behavioral model), a 4-bit subtractor, and a 4-bit multiplier.

Introduction

A full adder is a combinational circuit that adds three one-bit binary numbers - two operand bits and an incoming carry - and produces a sum bit and an outgoing carry bit. A full adder can be built hierarchically from two half adder modules and an OR gate, and multi-bit adders can, in turn, be built by chaining together several 1-bit full adders, with the carry propagating from one stage to the next (ripple-carry addition). In this experiment, Verilog HDL is used to design and simulate a half adder, a 1-bit full adder, and a 4-bit full adder in a hierarchical, bottom-up fashion, followed by a behavioral 32-bit full adder, a 4-bit subtractor, and a 4-bit multiplier. As in the previous experiment, the Cadence Incisive Enterprise Simulator (IES), invoked through NCLaunch, is used for compilation, elaboration, and simulation.

Part 1: Designing and Verifying the Half Adder

1. Create a lab directory and a simulation sub-directory under your work area:

```
mkdir lab3
cd lab3
mkdir simulation
cd simulation
```

2. Write Verilog code for a Half Adder using a structural/dataflow model. Save the file as halfAdder.v:

```
vi halfAdder.v
```

```
`timescale 1ns/1ps
module half_adder (
    input  a,
    input  b,
    output sum,
    output carry
);
    assign sum    = a ^ b;
    assign carry  = a & b;
endmodule
```

*Listing 1: halfAdder.v - dataflow description of a half adder.***3. Write Verilog test bench code for the above Half Adder. Save the file as halfAdder_tb.v:****vi halfAdder_tb.v**

```
`timescale 1ns/1ps

module half_adder_test;

    reg a;
    reg b;

    wire sum;
    wire carry;

    half_adder uut (
        .a(a),
        .b(b),
        .sum(sum),
        .carry(carry)
    );

    initial begin

        $monitor("Time = %0t | a = %b | b = %b | sum = %b | carry = %b",
            $time, a, b, sum, carry);

        // Test 1: 0 + 0
        a = 0;
        b = 0;
        #10;

        // Test 2: 0 + 1
        a = 0;
        b = 1;
        #10;

        // Test 3: 1 + 0
        a = 1;
        b = 0;
        #10;

        // Test 4: 1 + 1
        a = 1;
        b = 1;
        #10;

        $finish;

    end

endmodule
```

*Listing 2: halfAdder_tb.v - test bench that applies all four input combinations to half_adder and monitors sum and carry.***4. Simulate the Verilog files and verify the results. Include halfAdder_tb.v and halfAdder.v in the simulation.**

Part 2: Building the 1-bit and 4-bit Full Adder Hierarchically

5. Write Verilog code for a 1-bit Full Adder (FA) using the Half Adder (hierarchical structural model - two half adders and an OR gate combined to produce sum and carry-out). Save the file as fa1bit.v.
6. Write Verilog test bench code for the above FA. Save the file as fa1bit_tb.v.
7. Simulate the Verilog files and verify the results. Include fa1bit_tb.v, fa1bit.v, and halfAdder.v in the simulation.
8. Write hierarchical Verilog code for a 4-bit FA using the 1-bit FA (use a generate block to instantiate four 1-bit full adders with the carry rippling from one stage to the next). Save the file as fa4bit.v.
9. Write Verilog test bench code for the 4-bit FA. Save the file as fa4bit_tb.v.
10. Simulate the Verilog files and verify the results. Include fa4bit_tb.v, fa4bit.v, fa1bit.v, and halfAdder.v in the simulation.

Note: A generate block lets you instantiate a regular structure - such as four copies of a 1-bit full adder - inside a for-loop at compile time, rather than writing out each instance by hand. Remember to declare an internal carry vector (e.g., wire [3:0] carry_chain) to connect the carry-out of each stage to the carry-in of the next.

Part 3: Extending the Design - 32-bit Adder, Subtractor, and Multiplier

11. Create a 32-bit Full Adder (behavioral model, using the + operator on 32-bit vectors) and test it with a suitable test bench.
12. Write Verilog code for a 4-bit Subtractor and a 4-bit Multiplier, and verify each design using the same compile - elaborate - simulate flow described below.

Part 4: Launching NCLaunch and Creating the Work Library

The following steps walk through compiling, elaborating, and simulating a design using NCLaunch, using the Half Adder as the running example. Repeat the same procedure for the 1-bit FA, 4-bit FA, 32-bit FA, subtractor, and multiplier designs at each 'simulate and verify' step above.

Note: Figures 1 to 8 below are reference screenshots carried over from Experiment 3 (Simulation of Multiplexers), since the NCLaunch/SimVision workflow is identical. Your own windows will show halfAdder.v / halfAdder_tb.v (and later fa1bit.v, fa4bit.v, etc.) in place of the mux21.v / mux21_test.v shown in these screenshots.

13. We will use the Incisive Enterprise Simulator (IES) to verify that each design is working properly. Invoke the tool by typing 'nclaunch -new' in the terminal:

```
nclaunch -new
```
14. The NCLaunch window will appear. Click on 'Multiple Step'.

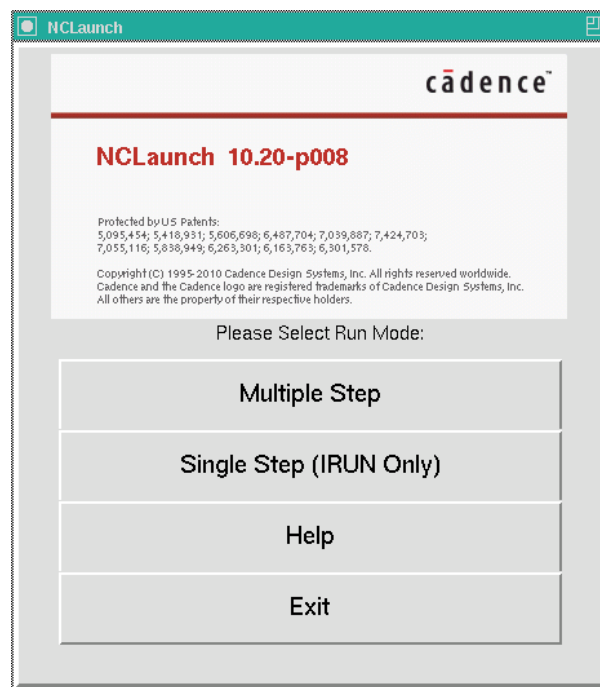


Figure 1: NCLaunch run-mode selection window.

15. The 'nclaunch: Open Design Directory' window will appear. Click on 'Create cds.lib File...'.

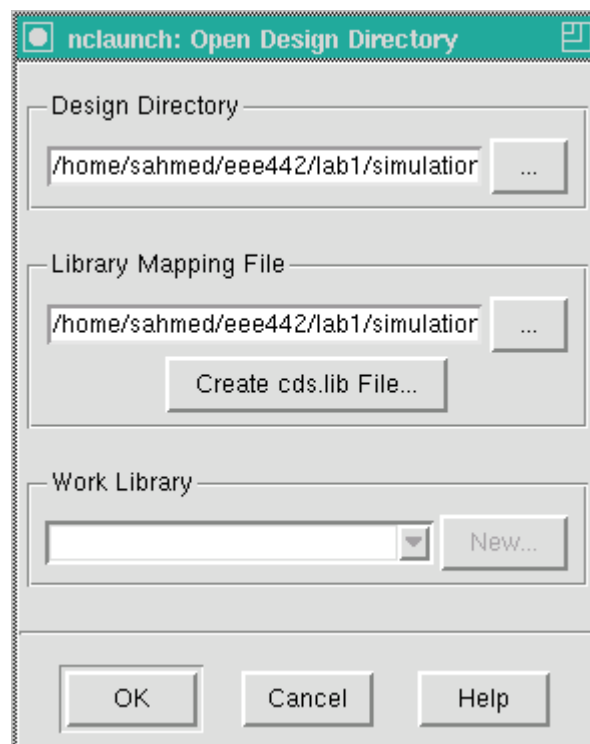


Figure 2: nclaunch Open Design Directory window, before the cds.lib file is created.

16. The 'Create a cds.lib file' window will appear. Click on 'Save'.

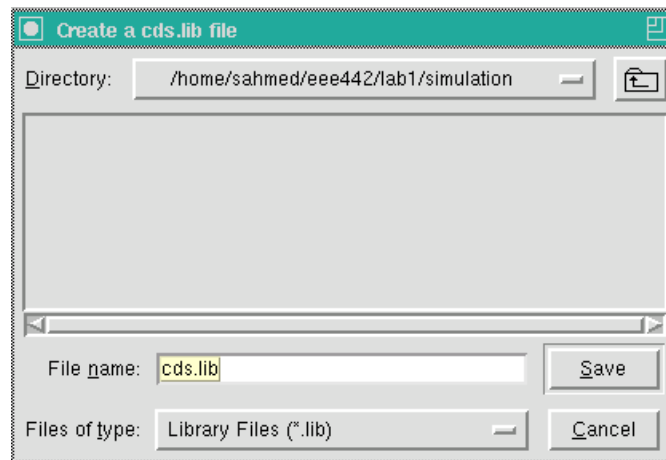


Figure 3: Create a cds.lib file window.

17. The 'New cds.lib File' window will appear. Select the third option, since we are using Verilog. Click on 'OK'.

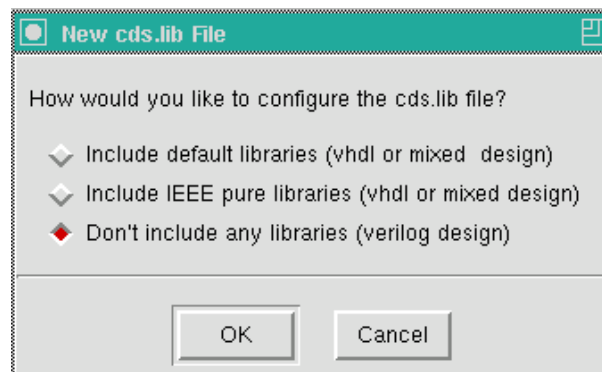


Figure 4: New cds.lib File window, with 'Don't include any libraries (Verilog design)' selected.

18. Click 'OK' in the 'nclaunch: Open Design Directory' window.

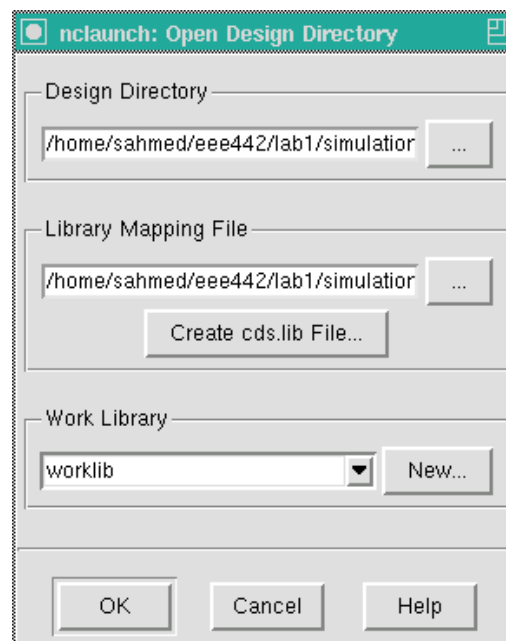


Figure 5: Open Design Directory window with the worklib now populated.

Part 5: Compiling the Design

19. In the NCLaunch window, we will see the design (halfAdder.v) and the test bench (halfAdder_tb.v) that we created in the simulation directory.

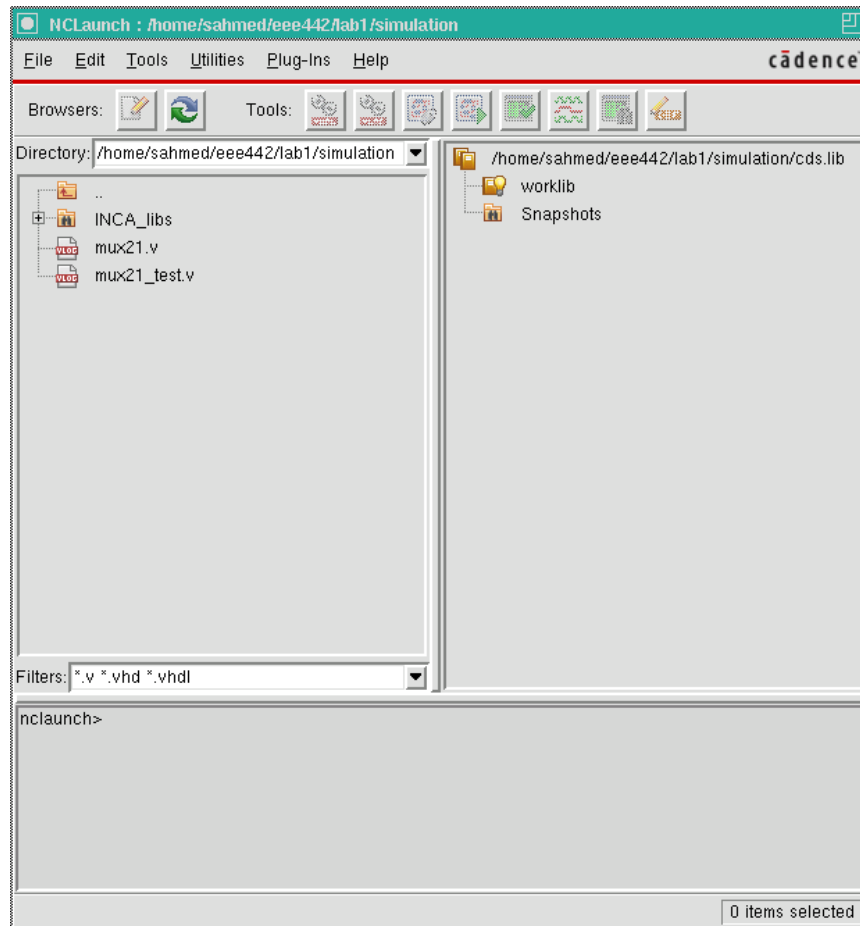


Figure 6: NCLaunch window showing the design and test bench files in the simulation directory (mux21.v / mux21_test.v shown here; you will see halfAdder.v / halfAdder_tb.v).

20. Now we will COMPILE the code to check its syntax and semantics. Select both the design and the test bench files, then click on ncvclog for Verilog designs.



Figure 7: NCLaunch toolbar and directory view for this experiment, with the ncvclog (compile) and other Tools icons visible.



Figure 8: NCLaunch toolbar, with the ncvclog (compile) icon highlighted.

21. Any error in the code will be reported in the 'Command Entry Window' at the bottom of NCLaunch. Clean all the errors in the design and/or the test bench before continuing.

Part 6: Elaboration - Building the Design Hierarchy

Elaboration constructs the design hierarchy and connects the signals between the design and the test bench.

22. Open the 'worklib' directory on the right side of the window and select the test bench, since it is the top module (it instantiates and contains the design module). Click on the 'launch elaborator' (ncelab).



Figure 9: NCLaunch toolbar, with the ncelab (elaborate) icon highlighted.

Part 7: Simulation - Running and Viewing the Waveforms

Simulation executes the elaborated design against the applied stimulus.

23. Open the 'Snapshots' directory on the right side of the window and select the worklib snapshot of the test bench. Click on 'launch simulator'.



Figure 10: NCLaunch toolbar, with the launch simulator icon highlighted.

24. A 'Console - SimVision' window will appear. This is used to perform simulation in command mode. Minimize this window, since we will be using GUI mode for simulation.
25. A 'Design Browser - SimVision' window also appears. There are many options in the Design Browser to analyze the design and for debugging. Select the test bench module on the left side of the window, then click on 'waveform'.

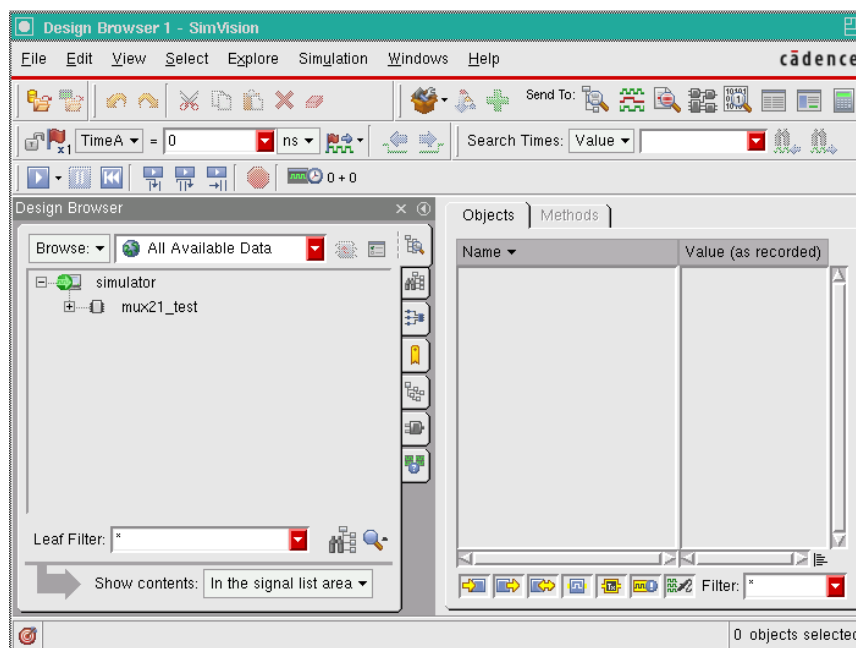


Figure 11: Design Browser - SimVision window, with the test bench module selected.

26. A 'Waveform - SimVision' window will appear. In the waveform window we will see the ports of the design. Click on 'run simulation' to start the simulation. Use the 'pause' button to interrupt or stop the simulation. Use options such as zoom in and zoom out to analyze the plot; the Waveform window offers many further options for analyzing the design and debugging.

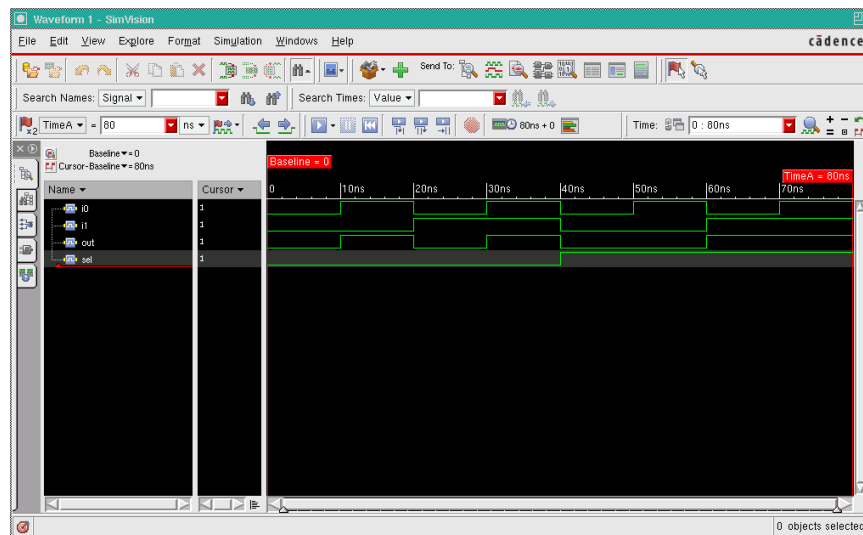


Figure 12: Waveform - SimVision window, showing simulated signals for the design under test.

27. After verifying the design, close the tools, and repeat Parts 4 to 7 for the next design in the hierarchy (half adder → 1-bit FA → 4-bit FA → 32-bit FA → subtractor → multiplier).

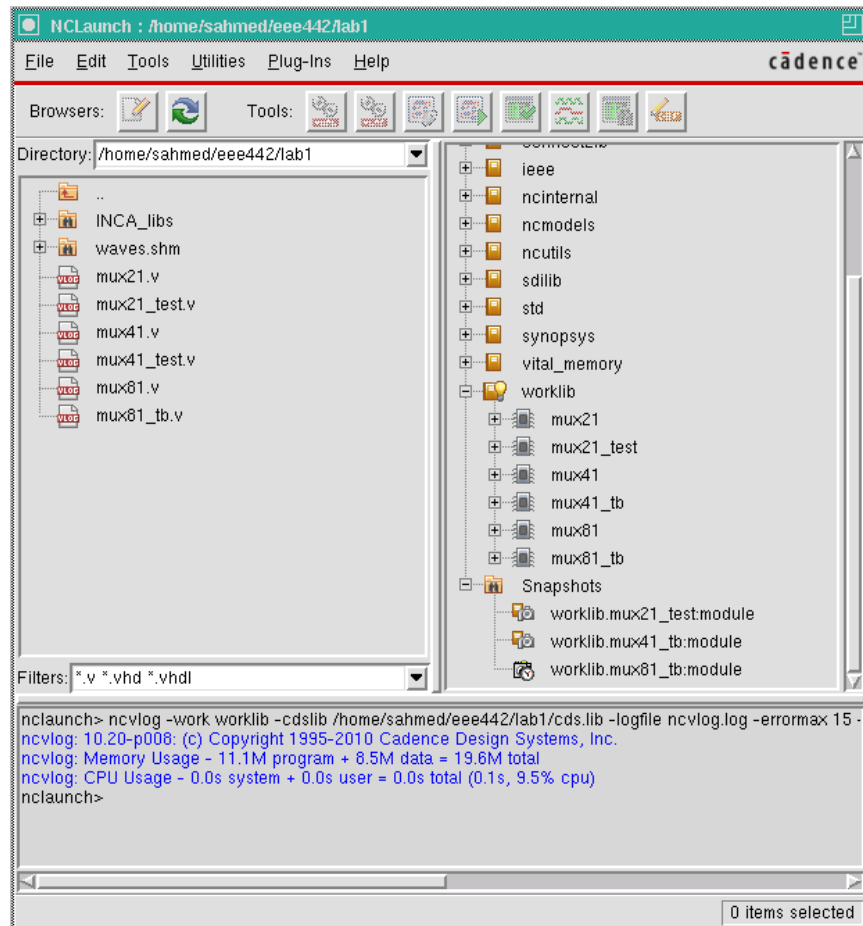


Figure 13: Example NCLaunch window after multiple designs have been compiled, elaborated, and simulated in the same worklib (illustrative, from Experiment 3) - by the end of this experiment your worklib and Snapshots will similarly list halfAdder, fa1bit, fa4bit, and the other designs.

Lab Report / Submission Checklist

1. Verilog source code and test bench listings for the half adder, 1-bit full adder, and 4-bit full adder (including the generate block).
2. Verilog source code and test bench listings for the 32-bit full adder, the 4-bit subtractor, and the 4-bit multiplier.
3. A screenshot of the NCLaunch window showing successful compilation (no errors in the Command Entry Window) for each design.
4. Waveform screenshots from SimVision for each design, clearly showing the inputs and outputs (e.g., a, b, sum, carry for the half adder; a, b, cin, sum, cout for the full adders).
5. A short discussion (150-200 words) explaining how the 4-bit full adder was built hierarchically from 1-bit full adders using a generate block, and how carry propagation was handled between stages.

Credits

This lab manual was prepared and formatted by:

Originated: EEE, UIU

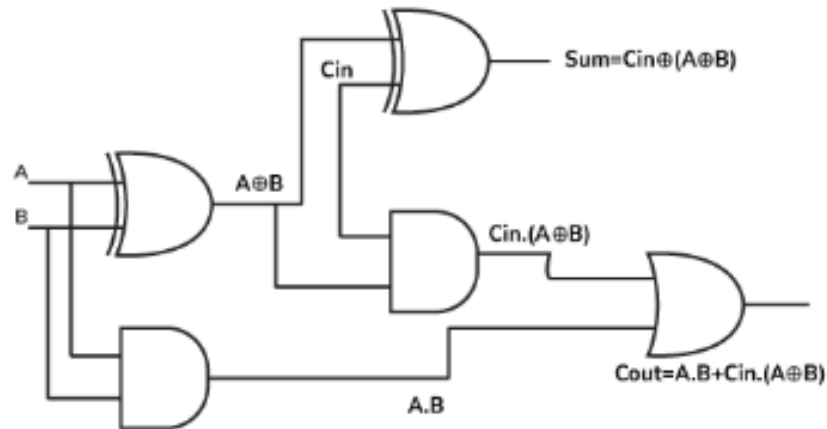
Sudipto Mondal

Lecturer, Department of Computer Science and Engineering (CSE)

United International University (UIU)

Email: sudiptomondal@cse.uiu.ac.bd

References for 4 bit Full Adder:



Inputs			Outputs	
A	B	C	Sum	Carry
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

